Understanding Our Research Paper

A complete study guide to

Quantifying Vendor Lock-In in Multi-Cloud Infrastructure: A Static-Analysis Lock-In Index and an Empirical Study of 677 Terraform Configurations

Deepak Soni (590012874) · Navneet Pant (590011569) · Mohd Toshif (590018042)

What this guide covers

- The whole story of the paper in plain language, then section by section.
- Every formula explained, with worked examples from our real data.
- How the data was collected and processed, step by step.
- How to read every table and graph, and what each one tells us.
- The statistics used (Mann–Whitney, Cliff's δ, Spearman, Monte Carlo), explained from scratch.
- How AI assistance was used, and how to talk about it honestly.
- How to re-run the whole analysis yourself, and the questions an examiner is likely to ask.

Numbers in this guide match the final version of the paper (after the classification fix described in Section 6.7).

Contents

1 How to use this guide **3**

2 The paper in one page **3**
2.1 The story . . . 3

3 Background concepts you must know **4**
3.1 Vendor lock-in . . . 4
3.2 Multi-cloud and portability . . . 4
3.3 Infrastructure as Code (IaC) and Terraform . . . 4
3.4 Kubernetes, PaaS and IaaS . . . 5
3.5 Egress . . . 5

4 The abstract, decoded **5**
4.1 The four research questions . . . 5

5 The Lock-In Index: dimensions, formulas and worked examples **6**
5.1 Why a rubric and not machine learning? . . . 6
5.2 The four dimensions . . . 6
5.3 Equation 1: the score of one resource . . . 6
5.4 Table I: the rubric . . . 6
5.5 Equation 2: the score of a configuration . . . 7
5.6 Lock-in mass: attributing lock-in to categories . . . 8

6 How the data was collected and processed **8**
6.1 Pipeline overview . . . 8
6.2 Step 1: choosing the repositories . . . 8
6.3 Step 2: finding root configurations . . . 8
6.4 Step 3: extracting resources . . . 8
6.5 Step 4: classification . . . 9
6.6 Exclusions and final numbers . . . 9
6.7 A classification bug we found and fixed . . . 9

7 Study design and the statistics, explained from scratch **9**
7.1 Architectural groups (for RQ3) . . . 9
7.2 Why medians, not means? . . . 10
7.3 Mann–Whitney U test . . . 10
7.4 Cliff's delta (δ): effect size . . . 10
7.5 Spearman's rank correlation (ρ) . . . 10
7.6 Monte Carlo sensitivity analysis (for RQ4) . . . 10

8 Results: how to read every table and graph **11**
8.1 Table II and Figure 2: LII by provider (RQ1) . . . 11
8.2 Figure 3: which categories drive lock-in (RQ2) . . . 12
8.3 Figure 4 and Table III: architectural groups (RQ3) . . . 12
8.4 Figure 5: sensitivity analysis (RQ4) . . . 13
8.5 Figure 6: exit egress cost . . . 14

9 Discussion, limitations and what you can (and can't) claim **14**
9.1 Practical implications (Section VII-A of the paper) . . . 14
9.2 No multi-cloud configurations . . . 15
9.3 Threats to validity: know these by heart . . . 15

10 How AI assistance was used **15**
10.1 What the AI assistant did 15
10.2 What you are responsible for 15

11 Reproduce the analysis yourself **16**

12 Likely examiner questions **17**

13 Glossary **18**

1 How to use this guide

Being able to defend a paper means you can explain *why* each choice was made, *what* each number means, and *where* the work is weak. This guide is built for that. Read it in order once, then use Section 12 (likely questions) to test yourselves.

An honest note before you start

This paper was produced with heavy help from an AI assistant, and the paper says so in its acknowledgment. That disclosure is required by IEEE and must stay. What makes the paper genuinely *yours* is not hiding that help. It is being able to explain, reproduce and defend every part of it. The fastest way to get there:

1. Read this guide fully, with the paper open beside it.
2. Re-run the analysis on your own laptop (Section 11). Then you have actually done the data collection yourselves.
3. Open 15–20 rows of `resources.csv` and check that each category makes sense to you.
4. Rewrite any paragraph of the paper that doesn't sound like you.

2 The paper in one page

2.1 The story

Problem Companies worry about *vendor lock-in*: once an app is built on one cloud (AWS, Azure, Google Cloud), moving it to another is expensive.

Gap Everyone talks about lock-in, but nobody measures it. Existing work is surveys and opinions. An architect cannot ask "how locked-in is *my* system?" and get a number.

Idea Cloud infrastructure is now written as code (Terraform). That code lists every cloud resource an app uses. If we give each kind of resource a lock-in score, we can compute a score for the whole app automatically.

Method We built the **Lock-In Index (LII)**: a 0–100 score. Each resource gets a score from four dimensions. A configuration's score is the average of its resources. We wrote a tool that computes it and ran it on 677 real Terraform configurations from 23 public GitHub repositories.

Findings (1) Most lock-in comes from "glue" (identity/permissions, networking, governance), not from the headline services. IAM alone is the biggest source. (2) Kubernetes-based designs are much less locked-in than designs built on managed services (median 25 vs 42). (3) The results hold even when we change the scoring rules randomly.

So what Teams can check lock-in automatically, like a security scanner. Recent exit-fee waivers and the EU Data Act reduced the *money* cost of leaving a cloud, but not the *technical* cost, which is what our index measures.

The 60-second pitch

"Vendor lock-in is one of the biggest worries in cloud computing, but it's always discussed qualitatively. We asked whether it can be measured. Because modern infrastructure is written as code, every cloud resource an application uses is listed in its Terraform files. We designed a Lock-In Index that scores each resource on four dimensions, API proprietariness, data lock, equivalence gap and egress exposure, and averages them into a 0 to 100 score. We applied it to 677 public Terraform configurations. The surprising result is that more than half the lock-in comes from glue like identity and networking, not from databases or serverless. Kubernetes-centric designs score much lower than managed-PaaS designs, and the rankings stay stable when we perturb the rubric. The main limitation is that we haven't yet validated the score against real migrations, and that's our next step."

3 Background concepts you must know

3.1 Vendor lock-in

Vendor lock-in means depending on one provider so much that switching becomes costly. In the cloud it happens because each provider has its own services, APIs and formats. Example: an app that stores data in AWS DynamoDB uses DynamoDB's own query API. Moving to Google Cloud means rewriting all that code for a different database, and also moving the data out.

3.2 Multi-cloud and portability

Multi-cloud means using more than one cloud provider. *Portability* means being able to move an application between providers. *Interoperability* means services from different providers working together. Petcu [8] draws this distinction, and our related-work section uses it.

3.3 Infrastructure as Code (IaC) and Terraform

Instead of clicking around a cloud console, engineers write files that describe the infrastructure they want, and a tool creates it. Terraform (by HashiCorp) is the most popular IaC tool. Its files end in `.tf` and use a language called HCL. Here is a short excerpt from one of the configurations in our dataset (`aws-samples/serverless-patterns`, folder `dynamodb-streams-lambda-terraform`):

```
resource "aws_dynamodb_table" "dynamodb_table_users" {
  name         = "UsersIds"
  hash_key     = "UserId"
  ...
}

resource "aws_lambda_function" "lambda_dynamodb_stream_handler" {
  function_name = "process-usersids-records"
  role          = aws_iam_role.iam_for_lambda.arn
  runtime       = "nodejs24.x"
}
```

Key terms:

- **Resource**: one piece of infrastructure, declared as `resource "TYPE" "NAME"`. In `aws_dynamodb_table`, the part before the first underscore (`aws`) is the **provider**, and the rest names the service.
- **Provider**: the plugin that talks to a cloud. `aws_` = AWS, `azurerm_` / `azuread_` / `azapi_` = Azure, `google_` = Google Cloud.

- **Module**: a reusable folder of Terraform code, called with `module "x" { source = "./path" }`.
- **Root configuration**: a folder that is deployed directly (for example an example app). This is our unit of analysis: one root configuration gets one LII score.

3.4 Kubernetes, PaaS and IaaS

- **IaaS** (Infrastructure-as-a-Service): virtual machines, disks, networks. Fairly similar across clouds.
- **PaaS** (Platform-as-a-Service): managed services like serverless functions (AWS Lambda), managed NoSQL (DynamoDB, Cosmos DB), workflow engines. Convenient, but each is unique to its provider.
- **Kubernetes**: an open-source system for running containers. Every major cloud offers a managed version (EKS, AKS, GKE), and the Kubernetes API is the same everywhere. That's why it is expected to reduce lock-in [6].

3.5 Egress

Egress is data leaving a cloud. Providers charge per gigabyte for it, so moving a large dataset out costs money. This is the most visible "exit fee".

4 The abstract, decoded

What the abstract says	What it means
"practitioners still judge it mostly by intuition"	The research gap: no metric exists.
"Lock-In Index (LII), a 0–100 score"	Our main contribution. Higher = more locked-in.
"21 service categories … four dimensions"	How each resource is scored (Section 5).
"677 root configurations (8,908 resource declarations, 844 distinct resource types) … 23 public repositories"	The size of our dataset.
"glue resources … 59.7% of all resources and 52.7% of the measured lock-in"	Finding 1 (RQ2): lock-in is spread through the scaffolding.
"median 41.9 … median 25.2; Cliff's $\delta = -0.84$, $p < 10^{-18}$"	Finding 2 (RQ3): managed-PaaS designs score far higher than Kubernetes ones. The difference is huge and not due to chance.
"median Spearman $\rho = 0.91$ over 1,000 trials"	Finding 3 (RQ4): rankings barely change when the rubric is perturbed.
"$850–1,100 at 10 TB to $48,000–80,000 at 1 PB"	The money cost of moving data out, at list prices.
"provider waivers and the EU Data Act"	Regulation reduced that money cost, but not the technical cost.

4.1 The four research questions

RQ1 How is lock-in distributed across configurations and providers? (*Descriptive*: what does the landscape look like?)

RQ2 Which service categories contribute most to lock-in? (*Where does it come from?*)

RQ3 Does the index separate architectural styles expected to differ? (*Does the index behave sensibly?*)

RQ4 How sensitive are results to the rubric's weights and scores? (*Can we trust it, given the scores are judgments?*)

5 The Lock-In Index: dimensions, formulas and worked examples

5.1 Why a rubric and not machine learning?

To train a model, you need labelled data: real migrations with measured costs. That data doesn't publicly exist. A *rubric* (a scoring table) needs no training data, and every number can be inspected and argued about. The paper calls this "transparent": if a reviewer disagrees with a score, they can change it and re-run.

5.2 The four dimensions

Each *category* of resource (not each individual resource) gets four scores between 0 and 1:

Dimension	Question it answers	Low (0) vs high (1) example
A: API proprietariness	How unique to one provider is the interface?	S3-style object storage (many clones exist): 0.3. IAM policy language: 0.9.
D: Data/state lock	How hard is it to export what the resource stores?	A network subnet stores nothing: 0. A DynamoDB table full of data in a proprietary format: 0.8.
E: Equivalence gap	How far is the nearest equivalent elsewhere from a drop-in replacement?	A VM is a VM everywhere: 0.1. BigQuery has no exact equivalent: 0.6.
X: Egress exposure	How much bulk data would you have to move out?	An IAM role: 0. An object-storage bucket: 0.9.

5.3 Equation 1: the score of one resource

$$\mathrm{LII}(r) = w_A A_{c(r)} + w_D D_{c(r)} + w_E E_{c(r)} + w_X X_{c(r)} \tag{1}$$

Reading it aloud: "The lock-in of resource r is a weighted sum of the four dimension scores of its category $c(r)$." The weights are $w_A = 0.35$, $w_D = 0.25$, $w_E = 0.25$, $w_X = 0.15$. They add up to 1, so $\mathrm{LII}(r)$ is between 0 and 1. In tables we multiply by 100.

Worked example 1: a DynamoDB table

Category: Proprietary NoSQL, with $A = 0.9$, $D = 0.8$, $E = 0.7$, $X = 0.8$.

$$\mathrm{LII} = 0.35(0.9) + 0.25(0.8) + 0.25(0.7) + 0.15(0.8) = 0.315 + 0.200 + 0.175 + 0.120 = 0.81 \;\rightarrow\; \mathbf{81.0}$$

Worked example 2: an IAM role

Category: IAM/identity, with $A = 0.9$, $D = 0$, $E = 0.5$, $X = 0$.

$$\mathrm{LII} = 0.35(0.9) + 0 + 0.25(0.5) + 0 = 0.315 + 0.125 = 0.44 \;\rightarrow\; \mathbf{44.0}$$

An IAM role stores no data, but it is written entirely in AWS's permission language, so it still scores fairly high.

Why these weights? A has the largest weight (0.35) because rewriting code that calls proprietary APIs is usually the biggest migration cost. X has the smallest (0.15) because moving data is a one-time, predictable cost. Section VI of the paper shows it is modest. D and E sit in between.

5.4 Table I: the rubric

Table I in the paper lists all 21 categories with their A, D, E, X scores and resulting LII, sorted from highest to lowest. Things to notice and be able to explain:

- **Top:** Proprietary NoSQL (81.0) and Analytics/AI (77.5). Proprietary APIs, data in proprietary formats, no direct equivalents, and lots of data to move.
- **Middle:** IAM (44.0), serverless functions (40.5), messaging (48.5). Proprietary APIs but little stored data.
- **Bottom:** Managed Kubernetes (17.0) and Kubernetes-native resources (6.0). The Kubernetes API is the same on every cloud.
- **Relational DB (33.5) is lower than NoSQL (81.0):** PostgreSQL or MySQL on RDS can be moved to Cloud SQL with standard tools. DynamoDB cannot.
- **Raw ARM API (`azapi`, 50.5):** a special Azure provider that calls Azure's internal API directly. It is highly Azure-specific.
- **Other provider (37.5):** the fallback score for the 1.3% of resources that didn't match any rule. It is set near the middle so it doesn't bias results.

5.5 Equation 2: the score of a configuration

$$\mathrm{LII}(C) = \frac{100}{|R_C|} \sum_{r \in R_C} \mathrm{LII}(r) \tag{2}$$

Reading it aloud: "The LII of configuration C is 100 times the *average* resource score over all its provider resources R_C." $|R_C|$ means "the number of resources in C". $\sum$ means "add up".

Worked example 3: a real serverless configuration (C0064)

`aws-samples/serverless-patterns/dynamodb-streams-lambda-terraform` declares 5 resources:

Resource type	Category	LII
`aws_dynamodb_table`	Proprietary NoSQL	81.0
`aws_lambda_function`	Serverless functions	40.5
`aws_lambda_event_source_mapping`	Serverless functions	40.5
`aws_iam_role`	IAM/identity	44.0
`aws_iam_role_policy`	IAM/identity	44.0

$$\mathrm{LII}(C) = \frac{81.0 + 40.5 + 40.5 + 44.0 + 44.0}{5} = \frac{250}{5} = \mathbf{50.0}$$

Worked example 4: a real Kubernetes configuration (C0131)

`hashicorp/learn-terraform-provision-gke-cluster` declares a GKE cluster (17.0), a node pool (17.0), a network (19.0) and a subnetwork (19.0):

$$\mathrm{LII}(C) = \frac{17 + 17 + 19 + 19}{4} = \frac{72}{4} = \mathbf{18.0}$$

Compare with example 3: same idea (a small app), but 50.0 vs 18.0. That is exactly the RQ3 finding in miniature.

Why the average and not the sum? With a sum, a big configuration with 100 harmless subnets would score higher than a small one built on DynamoDB. The average describes the *character* of a configuration, not its size. We checked this: the correlation between size and LII is only $\rho = 0.13$ (weak).

5.6 Lock-in mass: attributing lock-in to categories

To answer RQ2 ("which categories cause the most lock-in across the whole corpus?"), we add up the scores of every resource in each category:

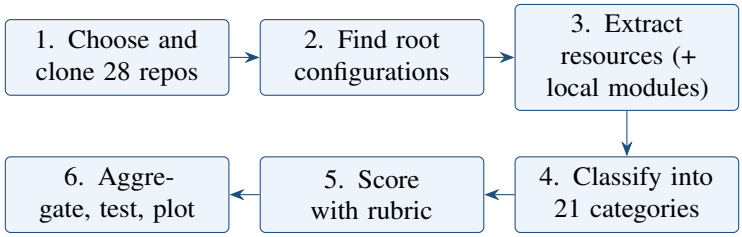
$$M_k = \sum_{C} \sum_{r \in R_C,\, c(r)=k} \mathrm{LII}(r), \qquad \mathrm{share}_k = \frac{M_k}{\sum_j M_j}$$

In words: M_k is the total lock-in "weight" of category k. It is large if the category has many resources, or high-scoring resources, or both. The share is its percentage of the total.

Example: IAM has 1,480 resources, each scoring 0.44, so $M_{\mathrm{IAM}} = 1480 \times 0.44 = 651.2$. Divided by the total mass of all categories, this gives IAM's 21.1% share.

6 How the data was collected and processed

6.1 Pipeline overview



6.2 Step 1: choosing the repositories

We needed public Terraform code covering all three big clouds and a range of architectures. We picked four kinds of source:

- **Official provider examples**: `hashicorp/terraform-provider-aws` and `hashicorp/terraform-provider-azurerm`.
- **Popular module collections**: `terraform-aws-modules/*` (VPC, EKS, RDS, Lambda, S3, DynamoDB, ECS, API Gateway), `terraform-google-modules/*`, Azure Verified Modules.
- **Vendor sample libraries**: Google's `terraform-docs-samples`, AWS `serverless-patterns`, Google's Cloud Foundation Fabric.
- **Tutorials**: HashiCorp's EKS, AKS and GKE "learn" repositories.

All 28 were cloned on Oct. 29, 2026. Each repository's exact version (commit SHA) is recorded in `repositories.csv`, so anyone can get exactly the same code. All are open source (Apache-2.0, MPL-2.0, MIT or MIT-0).

6.3 Step 2: finding root configurations

Repositories contain many folders. Only folders that are deployed directly count. The tool treats as roots: leaf folders under `examples/`, Cloud Foundation Fabric's `fast/stages/*`, each pattern folder with "terraform" in its name in `serverless-patterns`, each leaf sample folder in `terraform-docs-samples`, and the whole repo for the HashiCorp tutorials.

6.4 Step 3: extracting resources

For every `.tf` file in a root, the tool finds every line of the form `resource "TYPE" "NAME"` using a regular expression. If the folder calls a *local* module (`source = "../modules/x"`), the tool follows

it and counts that module's resources too, up to 8 levels deep. Modules downloaded from the Terraform registry are *not* followed (a limitation).

Then:

- The **provider** comes from the type prefix (`aws_`, `azurerm_`, `google_` and so on).
- **Utility providers** that create no cloud infrastructure (`random`, `null`, `tls`, `time`, `local`, `archive` and others) are removed.
- A configuration is kept only if it has **at least 3** provider resources. Tiny snippets aren't meaningful.

6.5 Step 4: classification

Each resource type goes into one of 21 categories by an *ordered* list of pattern rules. The first rule that matches wins. For example, a type containing `dynamodb`, `cosmosdb` or `firestore` becomes Proprietary NoSQL. One containing `iam_` or `role_assignment` becomes IAM/identity. One containing `vpc`, `subnet` or `route` becomes Network core. Resources from the Kubernetes and Helm providers form their own category. Only 1.3% of declarations matched no rule ("Other provider").

6.6 Exclusions and final numbers

- `aztfmod/terraform-azurerm-caf` was excluded: it builds resources from variable files, so its `.tf` files don't list them directly.
- Four repositories produced no qualifying configuration, mainly because they use registry modules.
- **Final corpus:** 677 configurations from 23 repos (AWS 191, Azure 248, GCP 238), 8,908 resources, 844 distinct types. Median size: 6 resources.

6.7 A classification bug we found and fixed

While preparing this guide, we checked individual rows of the dataset and found an error. Azure's managed Kubernetes resources (`azurerm_kubernetes_cluster` and `azurerm_kubernetes_cluster_node_pool`, 40 resources in total) had been put in the "Kubernetes-native" category (6.0) instead of "Managed Kubernetes" (17.0). The cause was a rule meant for the Kubernetes provider (`kubernetes_*`) also matching the word "kubernetes" inside Azure type names. We removed that rule and re-ran everything. The main changes: the Kubernetes-centric median moved from 24.2 to 25.2, and Cliff's δ from -0.87 to -0.84. No conclusion changed. The paper contains the corrected numbers.

If asked how you validated the classification

"We manually inspected individual rows of the resource table. That's how we caught a rule that misclassified Azure's AKS clusters as Kubernetes-native. We fixed it, re-ran the analysis, and the conclusions held. The full classification of every resource is in `resources.csv`, so anyone can audit it."

7 Study design and the statistics, explained from scratch

7.1 Architectural groups (for RQ3)

Every configuration was put into one group:

- **Kubernetes-centric** (47): uses managed Kubernetes or Kubernetes/Helm resources, and *no* proprietary PaaS.

- **Managed-PaaS-centric** (173): uses at least one of serverless functions, proprietary NoSQL, API/-workflow services or proprietary container platforms, and *no* Kubernetes.
- **IaaS/other** (457): everything else (networks, VMs, storage, IAM and so on).

No configuration mixed Kubernetes with PaaS.

7.2 Why medians, not means?

LII values aren't normally distributed (bell-shaped). Medians are robust to outliers. One extreme configuration can pull a mean but barely moves a median. That's also why we use *non-parametric* tests, which make no assumption about the shape of the data.

7.3 Mann–Whitney U test

Question it answers: "Do values in group A tend to be larger or smaller than values in group B, or is any difference just chance?"

How it works, intuitively: put all values from both groups in one list and rank them from smallest (rank 1) to largest. If the groups were the same, each group's ranks would be mixed evenly. If one group's ranks are mostly low, that group tends to have smaller values. U counts how often a value from one group beats a value from the other. z converts U into standard units, and p is the probability of seeing a difference this large if there were truly no difference.

Our result: $U = 642.0$, $z = -8.85$, $p = 9.0 \times 10^{-19}$. A p-value this tiny means the difference is essentially impossible to get by chance. (The usual threshold is $p < 0.05$.)

7.4 Cliff's delta (δ): effect size

A p-value says *whether* there is a difference, not *how big*. Cliff's δ measures size:

$$\delta = P(a > b) - P(a < b)$$

where a is a random value from group A and b from group B. It runs from -1 (every A is smaller than every B) to $+1$ (every A is larger).

Our result: $\delta = -0.84$ for Kubernetes-centric vs Managed-PaaS. If you pick one random configuration from each group, the Kubernetes one is lower about 92% of the time, since $(1 + 0.84)/2 = 0.92$, ignoring ties. Common rule of thumb: $|\delta| > 0.474$ is a "large" effect. Ours is very large.

7.5 Spearman's rank correlation (ρ)

Question it answers: "If I rank items by method 1 and by method 2, how similar are the two rankings?" $\rho = 1$ means identical order, 0 no relation, -1 reversed order. It compares *ranks*, not raw values.

We use it twice: to check that size doesn't drive LII ($\rho = 0.13$, weak), and in the sensitivity analysis.

7.6 Monte Carlo sensitivity analysis (for RQ4)

The worry: the rubric scores are our judgment. What if they're a bit wrong?

What we did: we repeated the whole scoring 1,000 times, each time with a randomly changed rubric:

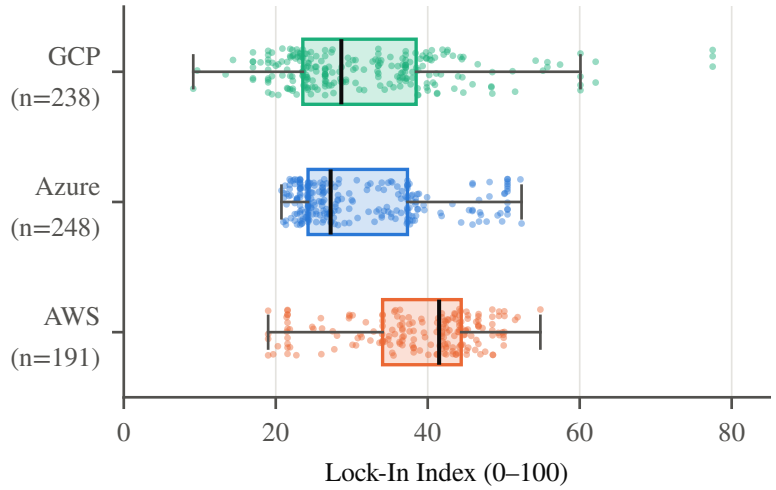
- **Weights** were drawn from a *Dirichlet distribution* centred on (0.35, 0.25, 0.25, 0.15). A Dirichlet distribution generates random sets of numbers that are positive and add up to 1, which is exactly what weights must do. "Concentration 4" controls the spread. It allows substantial variation, sometimes extreme.

- **Every category score** was shifted by a random amount between -0.15 and $+0.15$, kept within $[0, 1]$.

For each of the 1,000 trials, we compared the new ranking of configurations (and of categories) with the original, using Spearman's ρ. "Monte Carlo" just means "use many random trials to see what happens". The random seed was fixed at 42, so the results are reproducible.

8 Results: how to read every table and graph

8.1 Table II and Figure 2: LII by provider (RQ1)



How to read a box plot:

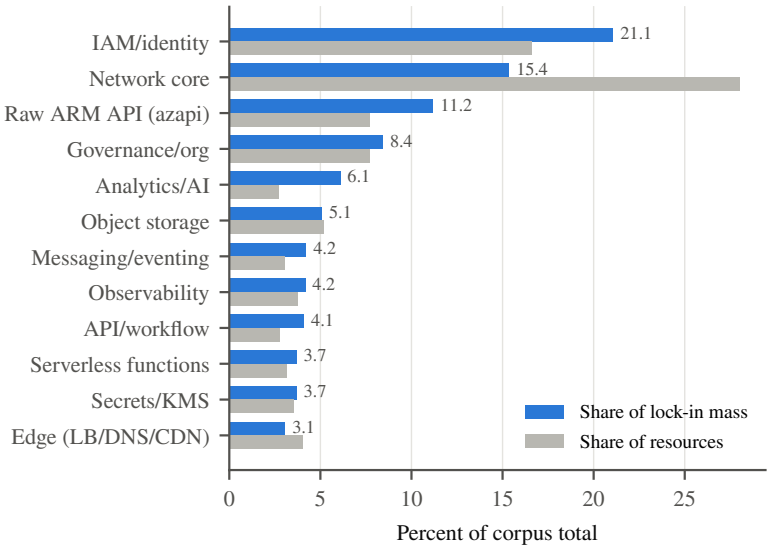
- The **thick line** inside the box is the **median** (half the configurations are below it).
- The **box** spans the **interquartile range (IQR)**: from the 25th percentile (Q1) to the 75th (Q3). The middle 50% of configurations lie inside it.
- The **whiskers** reach out to the most extreme values within $1.5\times$IQR of the box.
- The **dots** are individual configurations, jittered up and down so they don't overlap.

What it shows: AWS's box sits further right (median 41.5) than Azure (27.2) and GCP (28.6). Azure and GCP overlap almost completely.

How to interpret, carefully: this is *not* "AWS locks you in more". 90 of the 191 AWS configurations come from `serverless-patterns`, which is all Lambda, DynamoDB and API Gateway. The difference reflects *what the sampled code builds*, not the provider. The paper says this explicitly. If an examiner asks "which cloud is most locked-in?", the answer is: our data can't tell you, because the sample isn't balanced by architecture.

Statistics: AWS vs Azure $\delta = 0.43$; AWS vs GCP $\delta = 0.42$ (both $p < 10^{-13}$); Azure vs GCP $\delta = 0.00$, $p = 0.96$ (no difference). Overall: mean 33.5, range 9.1–77.5, and 27.2% of configurations score above 40.

8.2 Figure 3: which categories drive lock-in (RQ2)



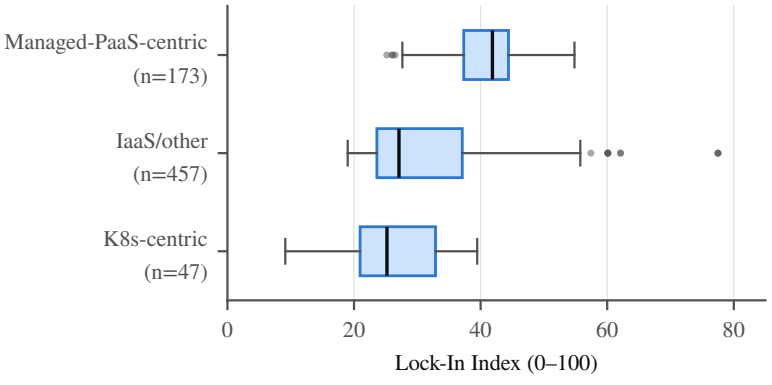
How to read it: for each category, the **grey bar** is its share of all *resources* and the **blue bar** is its share of total *lock-in mass*. The number at the end of each blue bar is the mass share.

- **Blue longer than grey:** the category contributes more lock-in than its frequency alone suggests, because each resource scores high. Examples: IAM, `azapi`, Analytics/AI.
- **Grey longer than blue:** common but low-scoring. Example: Network core (28.0% of resources, but only 15.4% of lock-in).

Key takeaways:

- **IAM is #1 at 21.1%.** IAM resources are 16.6% of all declarations and appear in 38.6% of configurations. Every role and policy is written in the provider's permission language and must be rewritten on another cloud.
- **Glue dominates:** IAM + network + governance + secrets + observability = 59.7% of resources and 52.7% of lock-in.
- High-scoring application services (NoSQL 81.0, Analytics/AI 77.5) matter per resource, but they are rare. NoSQL appears in only 3% of configurations.
- 26% of Azure resources use `azapi`, which shows how provider-specific code builds up when native Terraform support lags behind new Azure services.

8.3 Figure 4 and Table III: architectural groups (RQ3)

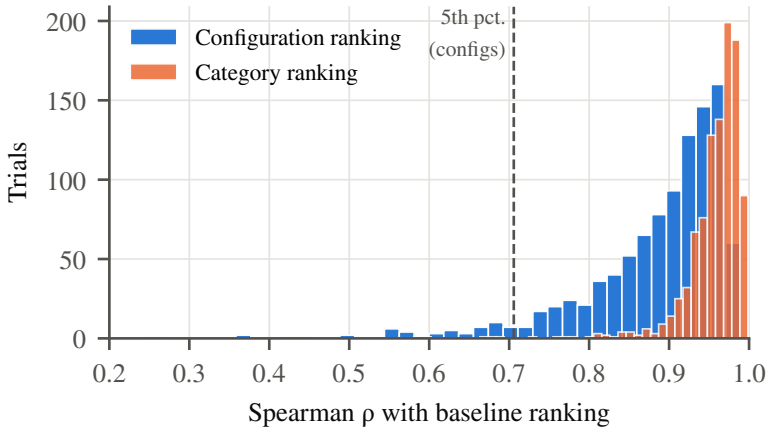


What it shows: Kubernetes-centric median 25.2 (mean 26.2, $n = 47$); IaaS/other 27.1 ($n = 457$); Managed-PaaS-centric 41.9 ($n = 173$). The boxes barely overlap. The dots to the right of the IaaS box are outliers (mostly analytics samples).

Interpretation: consistent with the common view that a shared orchestration API reduces lock-in. But:

1. Kubernetes is *not* lock-in-free. 25.2 is about 4 times the score of a pure Kubernetes resource (6.0), because the cluster sits on provider networking, IAM and node pools.
2. This is a **consistency check**, not independent proof. The groups are defined from the same categories the rubric scores, so it shows the index orders known styles sensibly. Real validation needs measured migrations.

8.4 Figure 5: sensitivity analysis (RQ4)

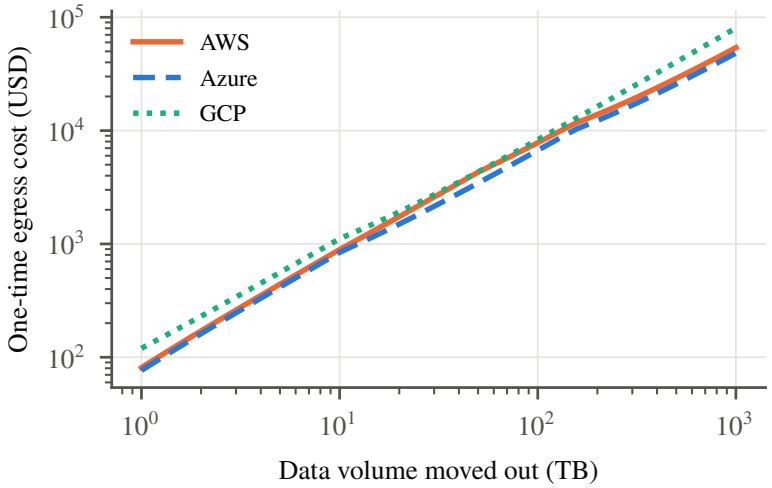


How to read a histogram: the horizontal axis is Spearman's ρ (how similar a perturbed ranking was to the original). Each bar's height is how many of the 1,000 trials fell in that range. **Blue** = configuration ranking, **orange** = category ranking. The dashed line is the 5th percentile for configurations: 95% of trials were to the right of it.

What it shows: both distributions are piled up near 1. Configuration ranking: median $\rho = 0.91$, 5th percentile 0.71. Category ranking: median 0.97, 5th percentile 0.90. The Kubernetes group had a lower median than the PaaS group in 99.7% of trials. The long thin left tail (down to 0.25) comes from rare trials where the random weights put almost everything on one dimension, which is an extreme case.

Interpretation: our conclusions don't depend on the exact rubric values. Individual mid-range scores can shift, but the overall picture holds.

8.5 Figure 6: exit egress cost



How to read a log–log plot: both axes are *logarithmic*. Each grid step is 10 times the previous one (1, 10, 100, 1000). This lets one chart show 1 TB and 1,000 TB together. A straight line on a log–log plot means cost grows roughly in proportion to volume. Small bends come from price tiers getting cheaper at higher volumes.

Worked example 5: moving 10 TB (10,000 GB) out of each cloud

AWS: first 100 GB free, then \$0.09/GB: $(10{,}000 - 100) \times 0.09 = 9{,}900 \times 0.09 = \$\mathbf{891}$.
Azure: first 100 GB free, then \$0.085/GB: $9{,}900 \times 0.085 = \$\mathbf{842}$.
Google Cloud (Premium Tier): first 1 GB free, next 1,023 GB at \$0.12, rest at \$0.11: $1{,}023 \times 0.12 + 8{,}976 \times 0.11 = 122.76 + 987.36 = \$\mathbf{1{,}110}$.

At 100 TB it is \$6,700–8,300, and at 1 PB \$48,000–80,000. Prices come from [15] (AWS, Azure) and [16] (Google). These are *list prices without waivers*.

Interpretation: for most companies, these amounts are small compared with paying engineers to rewrite proprietary code. That justifies giving X the lowest weight. Since 2024, Google, AWS and Microsoft all offer free egress to customers who are leaving [17], [18]. The EU Data Act [19] gives a legal right to switch and allows switching charges only until Jan. 12, 2027 [20]. These reduce the *money* barrier, but not the *technical* barrier (proprietary APIs, IAM, data formats) that dominates our index. *"Regulation has addressed the easy part of lock-in."*

9 Discussion, limitations and what you can (and can't) claim

9.1 Practical implications (Section VII-A of the paper)

1. Portability reviews should start with **IAM**, not just databases.
2. Adopting managed PaaS should be a **conscious trade-off**. One DynamoDB table can push a small configuration's score up sharply.
3. Kubernetes helps a lot, but the **cluster boundary** (networking, IAM, node pools) is where the remaining lock-in sits.
4. The analyzer could run in **CI pipelines** and flag pull requests that raise LII, like security scanners do [13].

9.2 No multi-cloud configurations

None of the 677 configurations used two providers. This is partly because these repositories are single-provider by design. It also fits the observation that "multi-cloud" in practice often means separate single-cloud stacks.

9.3 Threats to validity: know these by heart

Type	The threat	How we addressed it / what remains
Construct validity (are we measuring the right thing?)	Rubric scores are expert judgment, not measurements. Not validated against real migrations.	Sensitivity analysis shows robustness. Real-migration validation is the top future-work item.
Internal validity (is the measurement correct?)	Regex parsing, not full Terraform evaluation; `count`/`for_each` ignored; registry modules not expanded; `azapi_resource` is generic.	Only 1.3% unclassified; manual row checks (which caught and fixed the AKS bug); every classification published.
External validity (does it generalise?)	Examples and modules, not production systems. AWS sample leans serverless.	Stated openly. Future: run on TerraDS [7], tens of thousands of repos.

Do NOT claim

- "AWS is the most lock-in-heavy provider." (Sampling effect, see RQ1.)
- "The LII predicts migration cost." (Not validated yet.)
- "Kubernetes eliminates lock-in." (It reduces it; median 25.2, not 0.)
- "We analysed production systems." (We analysed public examples and modules.)

10 How AI assistance was used

The paper's acknowledgment states: *"The authors used an AI assistant (Claude, Anthropic) to help draft and edit portions of the manuscript text and analysis scripts. All analysis outputs were generated by the released code, and the authors take responsibility for the content."* IEEE policy requires AI-generated content to be disclosed in the acknowledgments, so this statement must stay.

10.1 What the AI assistant did

Being precise about this protects you. The AI assistant:

- Suggested research directions in cloud computing, and expanded the chosen topic (lock-in quantification) into research questions and a methodology.
- Searched for related literature, and checked each reference's details (authors, venue, DOI) against publisher and repository records.
- Wrote the analysis code (`lii.py`, `figs.py`, `export_dataset.py`), cloned the public repositories, ran the analysis, and generated the figures.
- Drafted the paper text in IEEE LaTeX format, and produced the reference list, dataset package and this guide.
- Found and fixed a classification bug during later checking (Section 6.7).

10.2 What you are responsible for

- Choosing the topic and directing the work.

- **Understanding and verifying** every claim, number and reference. The AI can make mistakes, as the bug shows.
- Making the final decisions: what to keep, what to change, where to submit.
- Being able to explain and reproduce the work. That is what this guide is for.

If asked "Did you use AI?"

"Yes, and we disclosed it in the acknowledgment as IEEE requires. We used Claude as a research assistant: it helped us find literature, write and run the analysis code, and draft the text. We chose the topic, checked the references, re-ran the analysis ourselves, and reviewed every result. We can walk you through any formula or number in the paper."

Only say "we re-ran the analysis" and "we checked the references" once you actually have. Section 11 shows how, and it takes under an hour.

11 Reproduce the analysis yourself

Everything you need is in `LockIn_Dataset.zip`. On a Mac (Terminal):

1. Set up a folder and Python.

```
mkdir lockin && cd lockin
unzip ~/Downloads/LockIn_Dataset.zip
python3 -m pip install matplotlib
mkdir corpus
```

2. Clone each repository at its exact commit. `tables/repositories.csv` lists each URL and commit SHA. This script clones them all (needs `git`):

```
tail -n +2 LockIn_Dataset/tables/repositories.csv | while IFS=, read repo
    url sha lic n status; do
  dir=corpus/$(echo $repo | sed 's#/#_#')
  git clone -q --filter=blob:limit=200k $url $dir && git -C $dir checkout -q
    $sha
done
```

3. Run the analysis and make the figures.

```
python3 LockIn_Dataset/code/lii.py corpus out
python3 LockIn_Dataset/code/figs.py out/results.json figs
```

The first command prints the number of configurations (677), resources (8,908), per-provider statistics, category shares, the group test and the sensitivity results. It takes a few minutes because of the 1,000 trials. The second writes the five figures as PDFs into `figs/`. Because the random seed is fixed, your numbers should match the paper exactly.

4. Check the classification yourself. Open `tables/resources.csv` in Excel or Numbers. Filter by `category` and look at 15–20 resource types. Ask: does this belong here? If you find something wrong, fix the rule in `lii.py` and re-run.

12 Likely examiner questions

About the idea

Q1. What is the main contribution?
A measurable, transparent Lock-In Index computed automatically from Terraform code, plus the first empirical study using IaC to measure lock-in (677 configurations).

Q2. What is the research gap?
Lock-in is widely discussed [1]–[3] but only qualitatively (surveys, reviews). No metric exists that can be computed for a specific system.

Q3. Why Terraform?
It is the most popular multi-cloud IaC tool, it lists every resource declaratively, and lots of public code exists [7], [13].

About the method

Q4. Why these four dimensions?
They turn the lock-in criteria in the literature into scores: proprietary APIs, data that is hard to export, missing equivalents, and the cost of moving data.

Q5. Why is w_A the largest?
Rewriting code against proprietary APIs is usually the biggest migration cost. Egress is one-time and predictable.

Q6. Who decided the rubric scores, and how do you justify them?
They are our expert judgments based on how each service works (for example, PostgreSQL on RDS is portable; DynamoDB isn't). We don't claim they're exact. RQ4 shows the conclusions hold under random changes of ± 0.15 and random weights.

Q7. Why the mean instead of the sum?
So the score reflects how locked-in a configuration is, not how big it is. Size vs LII correlation is only 0.13.

Q8. What is a root configuration?
A folder that is deployed directly, such as an example or sample app. It is our unit of analysis.

Q9. How do you handle modules?
Local modules are followed recursively and their resources counted. Registry modules aren't expanded, which is a limitation.

Q10. How accurate is the classification?
Only 1.3% of resources were unclassified. We checked rows manually, found and fixed one rule error (Azure AKS), and published every classification for audit.

About the results

Q11. What's the most surprising finding?
IAM, not databases or serverless, is the biggest single source of lock-in (21.1%). Glue categories make up more than half.

Q12. Does AWS lock you in more?
Our data can't say. The AWS sample is skewed toward serverless patterns. The difference reflects what the code builds, not the provider.

Q13. Explain Cliff's $\delta = -0.84$.
Pick one random Kubernetes-centric and one random PaaS-centric configuration: the Kubernetes one scores lower about 92% of the time. It is a very large effect.

Q14. What does $p = 9.0 \times 10^{-19}$ mean?
If there were really no difference between the groups, the chance of seeing a difference this large would be essentially zero.

Q15. Is RQ3 a validation of the index?
No, it's a consistency check. The groups come from the same categories the rubric scores. Real validation needs measured migrations.

Q16. What does a Spearman ρ of 0.91 mean in RQ4?
After randomly changing the rubric, the ranking of configurations was still 91% similar to the original (median trial). The conclusions are robust.

Q17. Why did no configuration use two clouds?
Partly because the repositories are single-provider by design. It also fits the idea that multi-cloud usually means separate stacks.

About egress and regulation

Q18. How did you calculate egress costs?
Tiered list prices from provider pricing [16] and a July 2026 price index [15], applied to volumes from 1 TB to 1 PB, without waivers. For example, AWS at 10 TB: 9,900 GB $\times$ \$0.09 = \$891.

Q19. What does the EU Data Act change?
It gives customers the right to switch, caps notice at two months, and allows switching charges only until Jan. 12, 2027. It cuts the money cost, but not the technical cost.

About limitations and future work

Q20. What's the biggest weakness?
No validation against real migration effort. That is our top future-work item: port a reference app between clouds, measure hours and code changes, and compare with LII.

Q21. What would you do next?
(1) Validate against real migrations. (2) Learn the scores from documented migrations instead of judgment. (3) Automate cross-provider service-equivalence mapping. (4) Scale to TerraDS and to CloudFormation/Bicep.

Q22. Did you use AI?
See Section 10. Answer honestly, the way the acknowledgment does.

13 Glossary

API	Application Programming Interface: how code talks to a service.
Cliff's δ	Effect size: $P(a > b) - P(a < b)$, from -1 to $+1$.
Commit SHA	Unique ID of an exact version of a Git repository.
Dirichlet	Distribution that produces random positive numbers summing to 1.
Egress	Data leaving a cloud; usually charged per GB.
IaC	Infrastructure as Code: infrastructure defined in files.
IAM	Identity and Access Management: users, roles, permissions.
IQR	Interquartile range: from the 25th to the 75th percentile.
Lock-in mass	Sum of resource scores in a category across the corpus.

Mann–Whitney U	Non-parametric test of whether one group tends to be larger.
Median	Middle value when sorted.
Monte Carlo	Many random trials used to study variability.
Non-parametric	A test that doesn't assume normally distributed data.
PaaS	Platform-as-a-Service: managed services such as Lambda and DynamoDB.
p-value	Probability of a result this extreme if there is no real effect.
Regex	Regular expression: a text-matching pattern.
Root configuration	A Terraform folder that is deployed directly.
Rubric	A scoring table, here 21 categories $\times$ 4 dimensions.
Spearman ρ	Correlation between two rankings, from -1 to $+1$.
Terraform / HCL	Most popular IaC tool, and its configuration language.
TOSCA	OASIS standard for describing portable cloud applications.